PROBLÈME · ANNALES SPÉ · 4 HEURES

Flots de données

1. Flots de données paresseux

OCaml exclusivement. `let list_finite = [1; 2; 3]` et `let rec list_ones = 1 :: list_ones`.

1. Croquis en maillons de `list_finite` et `list_ones` en mémoire ; quelle région de la mémoire ?

Le code suivant provoque `Stack overflow during evaluation (looping recursion?)` :

```
let list_fibo : int list =
  let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
    a :: (list_fibo_with_internal_state b (a+b))
  in
  list_fibo_with_internal_state 0 1
```

1. Traduire *stack* ; expliquer l'erreur.
2. Avec `ocamlopt fibo.ml -o fibo` puis `./fibo` : l'une des étapes échoue-t-elle ?

`type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)`. Exemples : `let rec ones = Cons (1, fun () -> ones)` ; `fibo` défini par `fibo_with_internal_state a b = Cons (a, fun () -> fibo_with_internal_state b (a+b))`.

1. Constante `integers : int stream` (entiers naturels croissants).
2. `range (a:int) (b:int) : int stream` (entiers de a inclus à b exclu).
3. Toute suite (σ_n) de chaînes est-elle codée par un `string stream` ?
4. `hd (u:'a stream) : 'a`, avec `Failure "hd"` si `u` est vide.
5. `tl (u:'a stream) : 'a stream`, avec `Failure "tl"`.
6. `of_list (l:'a list) : 'a stream`.
7. `iter (f:'a -> unit) (t:int) (u:'a stream) : unit` : applique `f` aux `t` premières valeurs existantes.
8. `map (f:'a -> 'b) (u:'a stream) : 'b stream`.
9. `zip (w:'a stream array) : 'a array stream` : flot des tableaux $[u_n, v_n, \dots]$, vide dès que l'un des flots l'est.
10. `intertwin (u:'a stream) (v:'a stream) : 'a stream` : alterne les éléments de `u` et `v`, en continuant avec l'autre quand l'un est épuisé.

```
let rec product_wrong u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      let part1 = product_wrong (tl_begetter1 ()) u2 in
      let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
      Cons ((hd1, hd2), fun () -> intertwin part1 part2)
```

1. Montrer que `product_wrong u1 u2` ne termine pas toujours, et qu'il suffit de déplacer quelques mots pour le corriger.
2. Signification du `0` dans `return 0` (C) ou `exit 0` (OCaml).
3. Énoncer le théorème de Turing sur l'arrêt.
4. On admet une fonction `run : string -> int -> int` qui termine toujours et renvoie 0 si le code x termine sans erreur en moins de n étapes, 1 sinon. Construire ou réfuter l'existence de `filter (f:'a -> bool) (u:'a stream) : 'a stream` (éléments de `u` vérifiant `f`, dans l'ordre ; par exemple `filter is_negative ones` vaut `Nil`).

2. Estimer le cardinal d'un flot : HyperLogLog

Cardinal d'un flot = nombre de valeurs distinctes. Le premier bit activé $\varrho(h)$ d'un entier $h \in [\![0, 2^{62}-1]\!]$ est le plus grand i tel que les $i-1$ bits de droite sont nuls ; $\varrho(0) = 63$ (exemples : $\varrho(1) = 1$, $\varrho(6) = 2$, $\varrho(8) = 4$). Hachage par `String.hash`.

1. Écrire `ffs (h:int) : int`.

Deux algorithmes sur les t premiers termes : `cardinality_nf` remplit une table de hachage et renvoie son nombre d'éléments ; `cardinality_fm` garde $r = \max \varrho(H(u))$ et renvoie 2^r.

1. Comparer leurs complexités en espace.
2. Écrire `harmonic_mean (z:float array) : float` ($\text{Harm} = m(\sum \frac{1}{z_j})^{-1}$).

HyperLogLog ($m = 16$, $\alpha_{16} = 0{,}6731$) : le terme u va dans le sous-flot $i = H(u) \bmod m$, et $r_i = \max \varrho(H(u)/m)$ sur ce sous-flot ; estimation $s \simeq \alpha_m \cdot m \cdot \text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}})$.

1. Écrire `hll_update (r:int array) (x:string) : unit` (met à jour r avec le nouveau terme x).
2. Écrire `mean_estimate (r:int array) : float`.
3. Dans le code multi-fils donné (deux fils remplissent `r_u` et `r_v` par `hll_update` ; le fil principal calcule toutes les 5 secondes `Array.init m (fun i -> max r_u.(i) r_v.(i))` et affiche l'estimation), identifier les courses critiques et proposer un mécanisme de correction.

3. Chiffrement par flot de booléens

$X(\varphi, \psi) = (\varphi \vee \psi) \wedge (\neg\varphi \vee \neg\psi)$; règles de déduction naturelle en annexe.

1. Arbres de preuve de $X(\varphi,\varphi)\vdash\bot$ et $\bot\vdash X(\varphi,\varphi)$.
2. Arbres de preuve de $X(\bot,\psi)\vdash\psi$ et $\psi\vdash X(\bot,\psi)$.
3. $\oplus(x,y)$ désigne l'évaluation de $X(x,y)$. Justifier que $(x,y)\mapsto\oplus(\oplus(x,x),y)$ et $(x,y)\mapsto y$ sont égales.

LFSR de longueur m, germe $(b_i)_{0\leqslant i<m}$, connexion $C\subseteq[\![0,m-1]\!]$: $b_{n+m}=\bigoplus_{i\in C}b_{n+i}$ ($\oplus$ est associative et commutative ; somme vide $=F$). Germe : `bool array` ; connexion : `int list`.

1. Écrire `lfsr (g:bool array) (c:int list) : bool stream`.
2. Avec `type formula = Var of int | Not of formula | And of formula * formula | Or of formula * formula`, écrire `logical_map (phi:formula) (w:bool stream array) : bool stream` : le flot des évaluations $\varphi(b_n^{(1)},\dots,b_n^{(k)})$ (φ ne dépend que de $x_1,\dots,x_k$, `w` de longueur k).
3. Alice et Bob partagent k LFSR et une formule ψ_0. Alice envoie publiquement $X(\psi_0(b_n^{(1)},\dots,b_n^{(k)}),c_n)$. Bob peut-il retrouver (c_n) ?

4. Mots de Dyck et algorithme de Heap

`let rec dyck = let f (x,y) = "a" ^ x ^ "b" ^ y in Cons ("", fun () -> map f (product dyck dyck))` ; D est l'ensemble des mots de ce flot.

1. Donner une grammaire non contextuelle G avec $L(G)=D$.
2. Énoncer une propriété de G qui assure que `dyck` n'énumère jamais deux fois le même mot, et la démontrer.

Algorithme de Heap : Heap(k, t) : si $k=1$, afficher t ; sinon, pour $i=0$ à $k-1$: exécuter Heap($k-1$, t), puis si k est pair échanger $t[i]$ et $t[k-1]$, sinon échanger $t[0]$ et $t[k-1]$.

1. Nombre $A(k)$ d'affichages lors de Heap(k, t).
2. Montrer : après Heap(k, t), si k est impair, t est inchangé ; si k est pair, les k premières cases ont subi une permutation circulaire vers la droite.
3. Montrer que Heap(k, t) affiche chaque permutation des k premières cases (les autres inchangées).
4. Écrire `pivot (k:int) (i:int) (t:'a array) : unit` (lignes 6 à 9).
5. Écrire `permstream_of_array (t:'a array) : 'a array stream`, flot fini des permutations dans l'ordre de l'algorithme de Heap, économe en mémoire.